Big Data Analytics Project

Big-Data Driven Taxi Fare Prediction & Demand Hotspot Analytics

TEAM MEMBERS

- Aditya S — CB.AI.U4AID24104
- Ashwin Naresh M — CB.AI.U4AID24106
- Divya Rithanya S — CB.AI.U4AID24113
- Vivethasri C — CB.AI.U4AID24162

FACULTY IN-CHARGE

Dr. Sreeja B. P
Assistant Professor Sr.Gd.
School of Artificial Intelligence,
Coimbatore

Foundations: Problem, Objectives & Big Data Need

1. Problem Statement

- **Urban mobility:** 175M+ taxi hailing events with high volatility across 263 NYC zones.
- **Price uncertainty:** Fares change with congestion, tolls, trip kinematics, and weather.
- **Imbalance:** Drivers cluster in hubs while outer boroughs remain underserved.

2. Project Objectives

- **Fare prediction:** Ridge Regression trained on 26.8M historical trips.
- **Demand regimes:** K-Means ($K = 3$) for Low, Moderate, and High-demand hotspots.
- **Streaming:** Structured Streaming with 3-sec micro-batches & 1-hr sliding window.

3. Why Big Data?

- 175M+ raw trips exceed single-node memory and processing limits.
- 4.63M spatiotemporal bins require distributed shuffle and aggregation.
- Real-time scoring needs stateful, fault-tolerant execution engines.

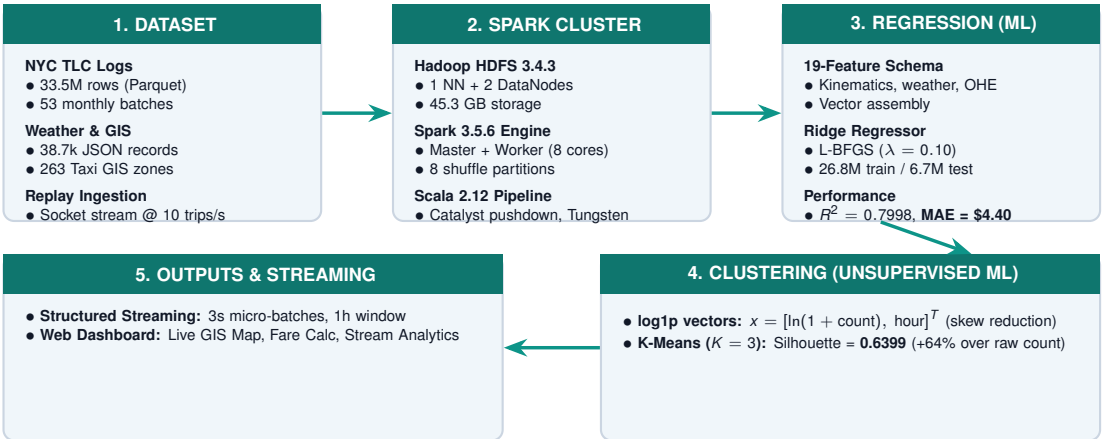
Dataset: Source, Size and the Five V's

Layer	Source	Format	Rows	Size	Period
Structured	NYC TLC Yellow & Green logs	Snappy Parquet	33,484,865	2.9 GB raw / 4.9 GB	2022-01 – 2026-05
Semi-struct.	Open-Meteo weather reanalysis	Nested JSON	38,712	18.4 MB	2022-01 – 2026-05
Spatial	NYC Taxi Zones GIS shapefiles	CSV / GeoJSON	263 zones	1.2 MB	Static

V	Highlight	Description
Volume	**33.5M rows**	128 MB HDFS blocks with 2× replication (45.3 GB) for parallel access.
Variety	**Multi-Format**	Columnar trips, JSON weather, and GIS spatial zones unified in Spark schemas.
Velocity	**3s Stream**	Structured Streaming with 1-hour sliding windows continuously updating demand.
Veracity	**Cleaned Data**	Filtered 0.1–100 mi and \$3–\$500 fares; chronological split prevents leakage.
Value	$R^2 = 0.7998$	\$4.40 MAE transparent fare pricing; hotspot analysis guides fleet routing.

Key attributes: Pickup/Dropoff datetimes, Distance, Duration, Speed, Passenger Count, LocationIDs, Boroughs, Temp, Precip, Wind.

System Architecture: End-to-End Pipeline



Big Data Execution: Spark Engine Optimizations

DataFrames, Catalyst & DAG Optimizations

- Built using DataFrame-based `spark.ml` API for optimized Catalyst logical plan generation.
- **Predicate Pushdown:** Filters unneeded partition files directly at the Parquet storage layer.
- **Projection Pruning:** Loads only active 19 model features into JVM off-heap memory.
- **Tungsten Engine:** Off-heap binary memory management eliminates GC and object creation overhead.

Partitioning & Memory Management

- Set `spark.sql.shuffle.partitions = 8` to align with cluster CPU cores.
- Strategic caching with `demandDF.cache()` across iterative K-means parameter grids.
- Cached prediction DataFrames prevent re-evaluating upstream ML pipelines during error slicing.
- Explicit `unpersist()` calls release worker executor memory after iteration.

Scala Engineering: Production Code Snippets (1/2)

Project Structure

```
surgesense-project/
|-- build.sbt
`-- src/main/scala/surgesense/
    |-- Phase1SplitAndEDA.scala
    |-- FeatureEngineering.scala
    |-- FinalLinearRegression.scala
    |-- DemandHotspotsV2.scala
    |-- FareErrorAnalysis.scala
    `-- StreamingInference.scala
```

Compile: `sbt package`

Submit:

```
spark-submit --master
  spark://spark-master:7077
  --deploy-mode client
```

Execution: Standalone cluster mode with 8 shuffle partitions and Tungsten off-heap.

1. Regularized Linear Regression Pipeline

```
// FinalLinearRegression.scala
val lr = new LinearRegression()
  .setLabelCol("fare_amount")
  .setFeaturesCol("scaled_features")
  .setMaxIter(100)
  .setRegParam(0.10)    // L2 (Ridge)
  .setSolver("l-bfgs") // quasi-Newton

val lrModel = lr.fit(trainDF)

lrModel.write.overwrite()
  .save("hdfs://namenode:9000/
         surgesense/models/...")
```

Scala Engineering: Production Code Snippets (2/2)

2. Stateful Sliding Window Stream

```scala
// StreamingInference.scala
val windowedStream = rawStream
  .withWatermark("pickup_datetime",
                 "10 minutes")
  .groupBy(
    window(col("pickup_datetime"),
           "1 hour", "5 minutes"),
    col("pickup_zone"),
    col("pickup_borough")
  )
  .agg(count("*").cast("double")
    .alias("pickup_count"))
```

3. Online K-Means Inference

```scala
val clustered = kmeansModel
  .transform(scaledStream)
  .withColumn(
    "demand_tier",
    when(col("cluster") === 2,
         "HIGH-DEMAND HOTSPOT")
      .when(col("cluster") === 0,
            "MODERATE DEMAND")
      .otherwise("LOW DEMAND")
  )
```

Spark ML Pipeline: Model Architecture & Tuning

Algorithm Specs (LR)

Parameter	Value
Solver	`l-bfgs`
RegParam (λ)	**0.10**
ElasticNet (α)	0.0
MaxIter	100
Standardization	True

$\alpha = 0.0$ corresponds to Ridge (L_2) regularization.

Hyperparameter Tuning (λ)

Metric	$\lambda = 0.01$	$\lambda = 0.10$
R^2 Score	0.79958	**0.79982**
RMSE	\$7.7935	**\$7.7889**
MAE	\$4.4118	**\$4.4027**
$\pm$\$2 Tol.	26.07%	**26.30%**

Evaluated on 6.7M hold-out test rows (Mar–May 2026). Tuned $\lambda = 0.10$ achieves optimal bias-variance tradeoff.

Hotspot Clustering (K-Means)

Clustering Spec	Config / Metric
Vector Space	$\mathbf{x} = [\ln(1 + \text{pcnt}), \text{hr}]^T$
Search Range	$K \in [2, 8]$
Optimal K	$K = 3$
Silhouette Score	**0.6399** (+64%)

Identifies 3 demand tiers: Low, Moderate and High-Demand Hotspots.

Results: Model Evaluation & Comparison (Baseline vs. Tuned)

Linear Regression: Baseline vs. Tuned Metrics

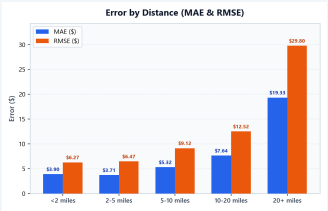
Metric	Original (λ=0.01)	Tuned (λ=0.10)
R^2 Score	0.799582	**0.799821**
RMSE	\$7.7935	**\$7.7889**
MAE	\$4.4118	**\$4.4027**
Tolerance ($\pm$\$2)	26.0683%	**26.3034%**

Key Performance & Business Insights

- L2 regularization with $\lambda = 0.10$ achieves the highest variance explained ($R^2 \approx 80\%$) across 6.7M test trips.
- Mean Absolute Error is constrained to \$4.40 citywide, giving drivers and riders reliable estimates.
- Over 1.76M predictions (26.30%) fall within $\pm$\$2.00 of the metered fare.
- Lower RMSE confirms effective mitigation of large-penalty outliers through L-BFGS convergence.

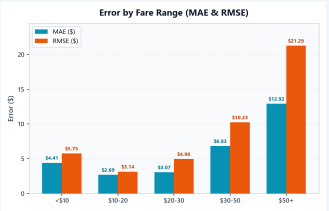
Results: Error Slicing (Dashboard Tab 3, 1/2)

Error by Distance Slice (MAE vs. RMSE)



- Minimum error at MAE $3.71 across core volume (2–5 miles, 85% of rides).
- Error rises to MAE $19.33 on long-haul trips (>20 miles) due to dynamic airport tolls.

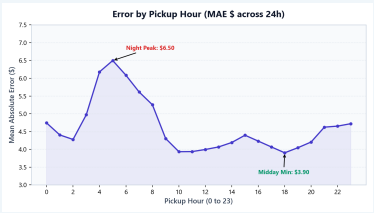
Error by Fare Tier (MAE vs. RMSE)



- Strongest fit at MAE $2.69 for standard $10–$20 fares.
- Higher RMSE of $21.29 on $50+ rides reflects long-distance surcharge variance.

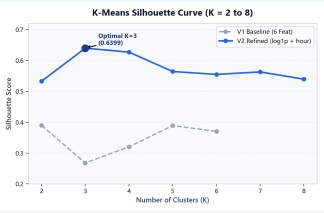
Results: Error Slicing & Clustering (Dashboard Tab 3, 2/2)

Error by Pickup Hour (MAE $ across 24h)



- Stable midday traffic (10:00–14:00) achieves the lowest error at $3.94.
- Early morning at 05:00 shows higher dispersion of $6.50 during fleet shift handover.

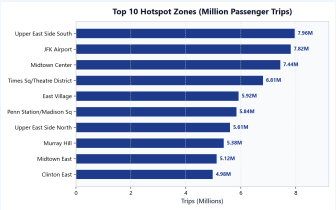
K-Means Silhouette Curve (V1 vs. V2 Refined)



- The log1p transformation boosts silhouette score from 0.3898 to 0.6399.
- A distinct global optimum at $K = 3$ supports 3 demand regimes.

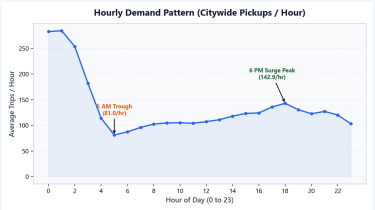
Results: Demand Hotspots & Spatial Dynamics (Tab 2, 1/2)

Top 10 Hotspot Zones (Million Trips)



- Highest concentrations: Upper East Side South (7.96M) and JFK Airport (7.82M).
- Major commercial transit points include Midtown Center (7.44M) and Times Square (6.81M).

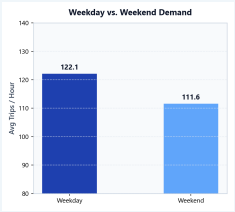
24-Hour Citywide Demand Pattern



- Citywide demand rises from a 05:00 trough (81.0/hr) steadily through the workday.
- Evening surge peaks at 142.9/hr at 18:00 during commercial rush-hour exit.

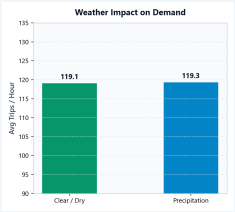
Results: Demand Hotspots & Spatial Dynamics (Tab 2, 2/2)

Weekday vs. Weekend Demand (Trips / Hour)



- Weekday volume of 122.1/hr is driven by repetitive Manhattan business transit.
- Weekend volume of 111.6/hr shifts toward leisure corridors and airports.

Weather Impact on Demand (Trips / Hour)



- Rain causes pedestrian modal substitution, maintaining trip rate at 119.3/hr.
- Overall demand remains inelastic at 119.1 vs 119.3/hr across weather conditions.

Critical Evaluation: Limitations & Engineering Roadmap

Technical Challenges & Limitations

- **Spatial imbalance:** Over 87% of rides concentrate in Manhattan and airport corridors.
- **Long-haul variance:** 20+ mile trips see elevated error (**MAE $19.33**) due to unobserved toll pricing.
- **Target leakage:** Post-trip variables such as `tolls_amount` cannot be used for pre-ride estimation.
- **Event-time skew:** Requires watermarking and state eviction to prevent unbounded state growth.

Conclusion & Phase 2 Roadmap

- Processed 33.5M records on Apache Spark and Hadoop HDFS.
- Linear fare regression achieved $R^2 = 0.7998$ and **MAE $\approx$ $4.40** .
- K-Means clustering achieved an optimal silhouette score of **0.6399** .
- **Phase 2:** Integrate OSRM toll engine, hierarchical fare models, and Apache Kafka streaming ingest.

Citations & Project Repository

References & Data Sources

[1] NYC Taxi & Limousine Commission (TLC), *TLC Trip Record Data*.
https://www.nyc.gov/site/tlc/about/tlc-trip-record-data.page

[2] B. S. B. D. Kumar and S. Hetharion, "Scalable Big Data Analytics and Fare Prediction for NYC Taxi Trips Using Distributed Computing and Machine Learning," *IJIES*, 2026.
https://international.arteii.or.id/index.php/IJIES/article/view/97

[3] K. Sundravadivelu et al., "Taxi Fare Prediction using Random Forest: A Data-Driven Approach," *Proc. ICISS*, 2025.
https://ieeexplore.ieee.org/document/11076256/

[4] S. Agarwal et al., "Taxi Fare Prediction Using Various Machine Learning Models," *IJFMR*, 2025.
https://www.ijfmr.com/research-paper.php?id=35839

[5] T. Ramadhan et al., "Application of K-Means Clustering for Urban Transportation Pattern Analysis," *JAIEA*, 2026.
https://ioinformatic.org/index.php/JAIEA/article/view/2237

[6] S. Mishra and S. Khetarpaul, "Identifying and Recommending Taxi Hotspots in Spatio-Temporal Space," *GeoInformatica*, 2025.
https://link.springer.com/article/10.1007/s10707-024-00524-w

Project Repository & Submission Link

GitHub Repository: https://github.com/surgesense-analytics/nyc-taxi-bigdata-platform
Deliverables uploaded: Presentation Deck (.pdf, .pptx), Comprehensive Project Report, Scala Code, and Docker Infrastructure.